

Ticketmaster API

Technical report — event discovery and ticketing backend

Go 1.22 · MongoDB Atlas · REST/JSON · OpenAPI 3.0.3
Repository: ticketmaster-backend · Branch: main · 31 July 2026

48 HTTP ENDPOINTS	8 COLLECTIONS	4 GO PACKAGES	3 ACCESS TIERS
----------------------	------------------	------------------	-------------------

1. Overview

The service is a REST API for event discovery and ticket sales, modelled on the public Ticketmaster Discovery API. It exposes a public catalogue of events, venues, attractions and classifications; lets registered users buy and cancel tickets; and gives administrators full control over the catalogue, user accounts and every booking in the system.

It is written in Go with no web framework — routing uses the standard library's `net/http.ServeMux`, which since Go 1.22 supports method and path patterns such as `PATCH /discovery/v2/events/{id}`. Persistence is MongoDB. The only third-party runtime dependencies are the MongoDB driver and `golang.org/x/crypto` for bcrypt.

2. Technology stack

CONCERN	CHOICE	RATIONALE
Language	Go 1.22	Static binary, no runtime to install, fast cold start on serverless
Routing	<code>net/http.ServeMux</code>	Go 1.22 added method + path-parameter patterns, removing the need for a router library
Database	MongoDB (Atlas)	Schema-flexible documents; TTL indexes expire tokens without a cleanup job
Passwords	bcrypt (x/crypto)	Deliberately slow hashing with a per-password salt
Documentation	OpenAPI 3.0.3 + Swagger UI	Spec embedded in the binary; served at <code>/docs</code>
Deployment	Docker, Render, Vercel	One handler, three endpoints

3. Project structure

The code follows the conventional Go layout: entrypoints under `cmd/`, implementation under `internal/`. The `internal/` prefix is enforced by the compiler — no module outside this one can import those packages, so the layout documents its own boundaries.

cmd/server/main.go	HTTP server endpoint (local, Docker, Render)
api/index.go	Vercel serverless endpoint – must sit at repo root
internal/	
config/	.env loading, environment lookup, DevMode
models/	domain types shared by storage and transport
store/	MongoDB persistence; all queries live here
httpapi/	routing, decoding, authorisation, responses
	openapi.yaml – embedded API specification

Dependencies run one way: `httpapi` → `store` → `models`, with `config` standing alone. The store knows nothing about HTTP; it returns sentinel errors (`ErrNotFound`, `ErrUnauthorized`, `ErrDuplicate`, `ErrSoldOut`, `ErrInUse`) which the HTTP layer maps to status codes. Changing a status code therefore never means editing the persistence layer.

4. Data model

COLLECTION	PURPOSE	NOTES
classifications	Genre taxonomy (segment, genre)	Referenced by events and attractions
attractions	Performers, teams	Referenced by events
venues	Physical locations	Referenced by events
events	Dated performances; ticket inventory	<code>ticketsSold</code> is server-owned
users	Accounts with a role	Unique index on email
bookings	Ticket purchases	Cancelled, never deleted, by users
tokens	Login sessions	TTL index — expire after 24 h
passwordResets	Single-use reset tickets	TTL index — expire after 1 h

A single Go struct serves as both the JSON wire format and the BSON document, keeping the API and the collections in step. The consequence is a rule the HTTP layer must honour: any field a client must not control — an id, or a derived counter such as `Event.TicketsSold` — has to be restored after decoding a request, because decoding alone would happily overwrite it.

5. API surface

48 operations across 26 paths, in three access tiers: **PUBLIC** needs nothing, **USER** needs a bearer token, and **ADMIN** needs a token belonging to an account with `role: admin`.

Service

ENDPOINT	TIER	DESCRIPTION
GET /	PUBLIC	Service metadata and endpoint index
GET /health	PUBLIC	Liveness; reports database connectivity as ok or degraded
GET /docs	PUBLIC	Interactive Swagger UI
GET /openapi.yaml	PUBLIC	Machine-readable specification

Discovery — events, venues, attractions, classifications

The same six operations exist for each of the four resources. Reads are public; every write requires an admin token.

ENDPOINT	TIER	DESCRIPTION
GET /discovery/v2/{res}	PUBLIC	Search, paged; filters vary by resource
GET /discovery/v2/{res}/{id}	PUBLIC	Fetch one
POST /discovery/v2/{res}	ADMIN	Create
PUT /discovery/v2/{res}/{id}	ADMIN	Partial update
PATCH /discovery/v2/{res}/{id}	ADMIN	Identical to PUT
DELETE /discovery/v2/{res}/{id}	ADMIN	Delete; refused while referenced

Authentication and account recovery

ENDPOINT	TIER	DESCRIPTION
POST /api/register	PUBLIC	Create a user account
POST /api/login	PUBLIC	Issue a 24-hour bearer token
POST /api/forgot-password	PUBLIC	Issue a single-use reset token
POST /api/reset-password	PUBLIC	Trade the token for a new password
POST /api/admin/register	ADMIN	Create an admin; requires the registration secret
POST /api/admin/login	PUBLIC	Admin sign-in; non-admins get the same 401 as a bad password
GET /api/admin/me	ADMIN	Validate a stored admin token

Bookings

ENDPOINT	TIER	DESCRIPTION
----------	------	-------------

POST /api/bookings	USER	Reserve tickets
GET /api/bookings	USER	Own bookings (plain array)
GET /api/bookings/{id}	USER	One own booking; another user's returns 404
DELETE /api/bookings/{id}	USER	Cancel; returns the booking, not 204
GET /api/admin/bookings	ADMIN	All bookings, filterable by user, event, status
GET /api/admin/bookings/{id}	ADMIN	Any booking
POST /api/admin/bookings/{id}/cancel	ADMIN	Cancel, keeping the audit record
DELETE /api/admin/bookings/{id}	ADMIN	Erase; releases tickets first

User administration

ENDPOINT	TIER	DESCRIPTION
GET /api/admin/users	ADMIN	List accounts; filter by keyword and role
GET /api/admin/users/{id}	ADMIN	Fetch one account
PUT / PATCH /api/admin/users/{id}	ADMIN	Edit, including role and password
DELETE /api/admin/users/{id}	ADMIN	Delete; revokes that account's tokens

6. Design decisions

6.1 Partial updates on PUT and PATCH

Both verbs share one handler that decodes the request body *over* the record already in the database, so a request needs only the fields that change. This departs from strict REST semantics, where PUT replaces the whole representation, and the reason is practical: strict replacement silently blanks every field an admin form forgets to send. Server-owned fields are restored after decoding, so a body containing `{"ticketsSold": 999}` or a forged `id` has no effect.

6.2 Referential delete guards

Deleting a record that others still point at would orphan data, so those deletes are refused with `409 Conflict` and a message naming the fix rather than performing a silent cascade:

DELETING	REFUSED WHILE
Event	confirmed bookings exist
Venue	any event is scheduled there
Attraction	any event still lists it
Classification	events or attractions still use it
User	the user holds confirmed bookings

6.3 Ticket inventory correctness

Reserving tickets is a single conditional update: the seat count is decremented only if the event is on sale and enough tickets remain, evaluated by the database in one atomic operation. Two buyers racing for the last pair of tickets therefore cannot both succeed. Cancelling returns the seats and is a safe no-op if repeated; deleting a still-confirmed booking releases its seats first, so an event can never be left overcounted.

6.4 Role assignment

A user's role is set by the server from the endpoint used to register, never read from the request body. Posting `{"role": "admin"}` to `/api/register` produces an ordinary user. Admin accounts come only from `/api/admin/register`, which requires a shared secret (`ADMIN_REGISTRATION_KEY`) compared in constant time; while that variable is unset the endpoint is disabled rather than open.

7. Security

CONCERN	MEASURE
Password storage	bcrypt hashes only; the plaintext is never stored and the hash never appears in a response
Credential probing	An unknown email and a wrong password return an identical 401
Admin discovery	A non-admin signing in at the admin endpoint gets the same 401, so admin emails cannot be identified

Account enumeration	<code>/api/forgot-password</code> returns the same 200 body whether or not the address exists
Reset tokens	256-bit from <code>crypto/rand</code> , single-use, one-hour expiry enforced in code as well as by a TTL index
Session hygiene	A completed reset revokes every existing session for that account
Privilege escalation	Roles are never taken from a request body; an admin cannot demote or delete their own account
Error disclosure	Internal failures are logged in full but returned as a generic message
Request limits	Bodies capped at 1 MiB; page size capped at 100
Secrets	Configuration comes from the environment; <code>.env</code> is git-ignored and excluded from Docker images

Password recovery is token-based by design. An earlier iteration reset a password given only an email address, which meant anyone knowing a customer's email could take over the account. The flow now requires a token that the server issues and the requester must present. Because no mail service is configured, the token is written to the server log; outside production it is also returned in the response so the flow can be exercised locally. That convenience does reveal whether an account exists, which is why it is disabled whenever `ENV` is anything other than an explicitly development value.

8. Verification

Behaviour was checked by exercising the running API against a scratch database, not by inspection alone. Scratch databases were dropped afterwards. The following properties were confirmed end to end:

- Admin registration rejects a wrong key (403) and is disabled without the secret (503).
- Posting `role: admin` to the public register endpoint yields `role: user`.
- A user token on an admin route returns 403; no token returns 401.
- A partial update changed two fields and preserved the other eight.
- Forged `id` and `ticketsSold` values in an update body were ignored.
- All five delete guards returned 409 while referenced, and 204 once the reference was removed.
- Cancelling a booking restored `ticketsSold` from 3 to 0 and was idempotent.
- Password reset: token issued, consumed once, rejected on reuse, superseded by a newer token, rejected when backdated past its expiry, and the pre-existing session was revoked.
- After restructuring the packages, 17 endpoint checks across all four packages reproduced pre-refactor behaviour exactly.
- The OpenAPI document validates as OpenAPI 3.0.3, and its operation list was diffed against the router: 48 of 48, nothing undocumented and nothing invented.

`go build`, `go vet`, `go test` and `gofmt` are clean across the module.

9. Deployment and operations

One handler is built by `httpapi.New()` and shared by three entrypoints: the local/container server, Render's native Go runtime, and Vercel's serverless functions. The Docker image is a multi-stage build — a static CGO-free binary on Alpine, running unprivileged, with `ca-certificates` installed because Atlas connections are TLS and fail without them.

VARIABLE	PURPOSE
MONGO_URI	Connection string; never committed
DB_NAME	Database name
PORT	Listen port (default 8080)
ENV	Anything but an explicit dev value is treated as production
ADMIN_REGISTRATION_KEY	Secret required to create an admin account
ADMIN_NAME / EMAIL / PASSWORD	Optional bootstrap admin, seeded at startup

Startup is deliberately tolerant: an unreachable database does not stop the process, it only makes individual requests fail, and `/health` reports `degraded`. A container therefore rides out a brief database outage instead of restart-looping. Index creation and admin seeding are best-effort and log a warning on failure.

```
make run      # start the API
make build    # static binary, same flags as the Dockerfile
make check    # vet + tests + a gofmt gate for CI
make docker   # build the image
```

10. Known limitations

- **No rate limiting.** Login and password-reset endpoints can be called repeatedly; brute-force and reset-spam protection is not implemented.
- **Login tokens are predictable in structure.** Sessions use MongoDB ObjectIDs, which embed a timestamp and counter. Reset tokens already use `crypto/rand`; session tokens should follow.
- **No password strength rules on registration.** A minimum length is enforced when resetting a password but not when creating an account.
- **No automated test suite.** Verification was done by exercising the running API; only the environment-gate logic has a unit test. Handler and store tests would make regressions cheaper to catch.
- **No email delivery.** Reset tokens go to the server log rather than to the account holder.
- **Specification drift is possible.** The OpenAPI document is written by hand rather than generated from the handlers, so it must be updated alongside route changes.
- **An unused dependency remains declared** — `golang-jwt/jwt` is in `go.mod` but never imported; the design uses opaque tokens, not JWTs.

11. Summary

The service implements the full lifecycle it set out to cover: a public catalogue, user registration and authentication, ticket purchase with race-safe inventory, token-based password recovery, and administrative control over every resource, account and booking. The codebase is organised along the conventional Go layout with a one-way dependency graph, documented at package level and through an embedded OpenAPI specification served from the API itself. The security posture addresses the common account-level risks — credential storage, enumeration, privilege escalation and session invalidation — while the limitations above, chiefly rate limiting and automated tests, are the natural next steps.